

Name: **Ruiqi Liu**

Student ID: **2434079**

Edge Impulse Link: <https://studio.edgeimpulse.com/public/1007395/live>

Part 1

```
Starting inferencing, press 'b' to break
Starting inferencing in 2 seconds...
Sampling...
INFO: HW RFFT failed, FFT size not supported. Must be a power of 2 between 32 and 4096, (size was 16)
Timing: DSP 79 ms, inference 569 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.996094
  lift: 0.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 88 ms, inference 532 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.996094
  lift: 0.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 88 ms, inference 562 us, anomaly 0 ms, postprocessing 66 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 84 ms, inference 559 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
```

As shown, both idle and lift states are detected.

Part 2

Question 1: Model architecture and ensemble flow

Draw a complete diagram of the full ensemble pipeline. Your diagram should include enough detail for someone to reconstruct the model architecture from your drawing.

Include the following details:

The input window size and number of input neurons. In this notebook, each input window has 100 time steps and 6 IMU features, so the flattened input dimension is 600.

The three input branches: raw input, standard-scaled input, and min-max-scaled input.

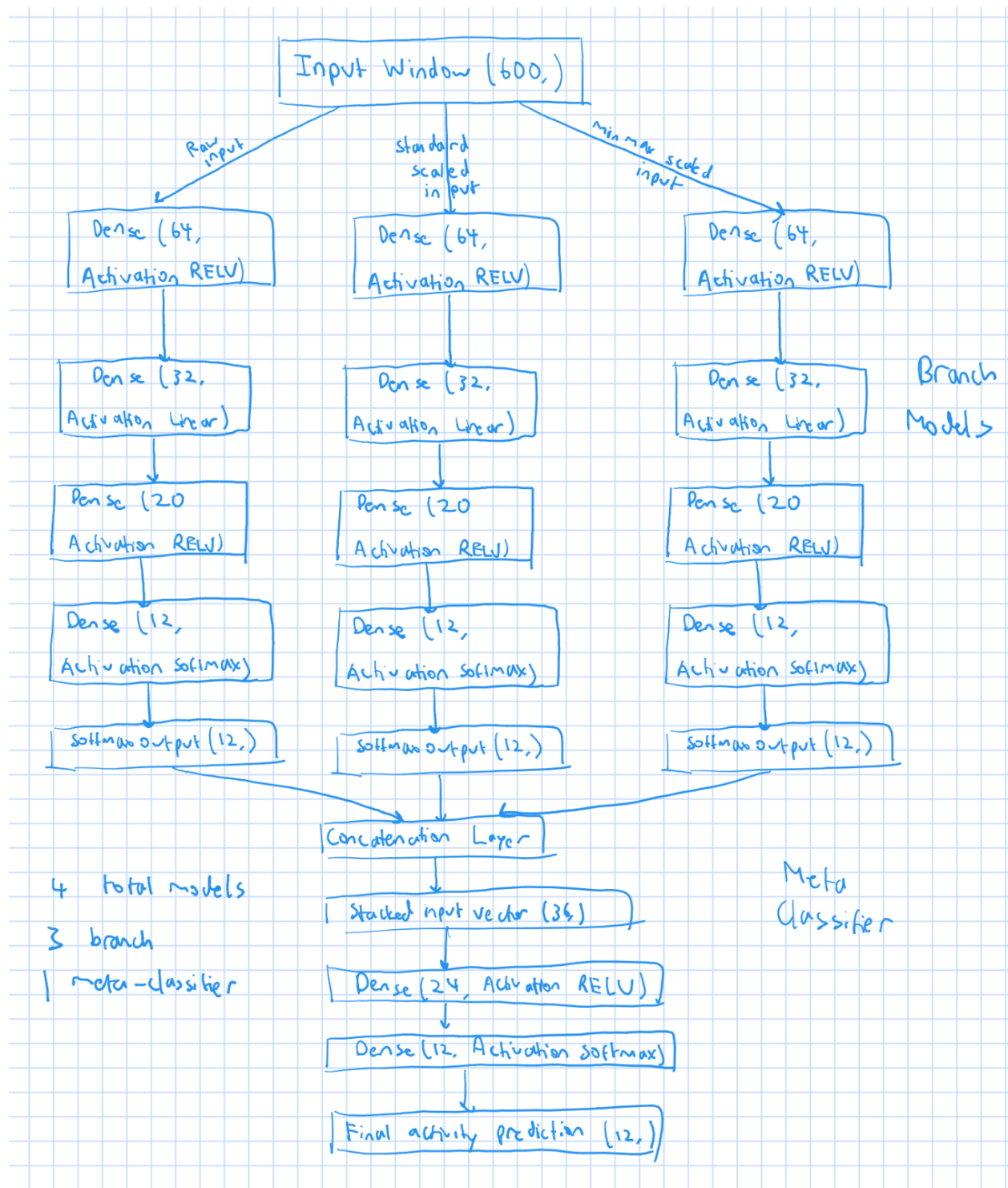
The encoder architecture in each branch, including the number of hidden layers, number of neurons in each layer, and activation function used in each layer.

The classifier architecture in each branch, including the number of input neurons, hidden-layer neurons, output neurons, and activation functions.

The number of models used in the ensemble and how they are connected.

The size of each branch output and how the three outputs are concatenated before being passed into the stacked meta-classifier.

The stacked meta-classifier architecture, including input size, hidden layer size, output size, and activation functions.



Question 2: Pruning and QAT methodology

Explain how this notebook compresses the models for TinyML deployment.

In your answer, describe:

How pruning is applied and what sparsity schedule is used.

Why the pruning wrappers are stripped before QAT.

How QAT is applied after pruning.

Why the notebook uses an additional mask-enforcement step during and after QAT instead of using only a simple pruning-then-QAT pipeline.

How the final model is converted to an int8 TFLite model and why representative data is needed.

How pruning and int8 quantization help when deploying to a resource-constrained device such as the Arduino Nano 33 BLE Sense.

The notebook applies magnitude-based pruning which follows a polynomial decay sparsity schedule from 10% to 80% over 500 training steps (5 epochs). After this pruning, the notebook calls `strip_pruning()` to remove the pruning wrappers, which is important because the quantization later on is not meant to work with the pruning wrappers on. After stripping the pruning wrappers, quantization aware training is applied using `quantize_model()`, which goes through 5 epochs of fine-tuning.

This notebook uses mask-enforcement after stripping. Without mask enforcement, the QAT's simulated noise may cause weights to drift. Mask enforcement forces the weights to respect the original pruned mask at the end of every batch and epoch.

The final model is then converted into an int8 TFLite model using `tf.int8`. Representative data is needed here for calibration after converting everything to int8; the converter uses the samples to correctly calculate the scales and zero-points needed to map everything to int8.

Pruning and int8 quantization help when deploying to a resource constrained device because pruning greatly reduces the number of parameters, and int8 quantization makes each parameter 1 byte instead of 4 (float32). This allows it to be deployed on a chip that doesn't have that much memory, like the Nano 33 BLE Sense. Also, int8 quantization reduces the complexity of computations, leading to a lower power consumption.

Question 3: Arduino deployment and real-world behavior

Deploy the quantized model using the provided Arduino sketch for this lab. Place the Arduino Nano 33 BLE Sense flat on a stable surface, open the serial monitor, and observe the predicted activity labels.

In your answer, include:

A screenshot of the serial monitor output.

A short explanation of whether the output is acceptable for the resting board condition.

Any issues you observe, such as unstable predictions, unexpected labels, sensor noise, scaling mismatch, or mismatch between the training data and the real board orientation.

One or two practical changes that could improve deployment reliability.

```
Raw model scores: 0.0000, 0.5000, 0.0000, 0.0000, 0.0000, 0.0000, 0.5000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
Standard-scaled model scores: 0.0000, 0.0000, 0.5117, 0.0000, 0.0000, 0.0000, 0.0156, 0.0000, 0.0000, 0.4609, 0.0000, 0.0078
Min-max-scaled model scores: 0.0195, 0.0078, 0.3398, 0.0000, 0.3398, 0.1914, 0.0781, 0.0000, 0.0000, 0.0000, 0.0000, 0.0195
Meta-classifier scores: 0.0039, 0.0312, 0.3164, 0.0000, 0.0156, 0.0039, 0.3555, 0.0000, 0.0000, 0.0430, 0.2227, 0.0078
-----
Final prediction: Frontal elevation of arms
Class index: 6
Confidence score: 0.3555
-----
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.5000, 0.0000, 0.0000, 0.0000, 0.0000, 0.5000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
Standard-scaled model scores: 0.0000, 0.0000, 0.5117, 0.0000, 0.0000, 0.0000, 0.0156, 0.0000, 0.0000, 0.4609, 0.0000, 0.0078
Min-max-scaled model scores: 0.0195, 0.0078, 0.3398, 0.0000, 0.3398, 0.1914, 0.0781, 0.0000, 0.0000, 0.0000, 0.0000, 0.0195
Meta-classifier scores: 0.0039, 0.0312, 0.3164, 0.0000, 0.0156, 0.0039, 0.3555, 0.0000, 0.0000, 0.0430, 0.2227, 0.0078
-----
Final prediction: Frontal elevation of arms
Class index: 6
Confidence score: 0.3555
-----
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.5000, 0.0000, 0.0000, 0.0000, 0.0000, 0.5000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
Standard-scaled model scores: 0.0000, 0.0000, 0.5117, 0.0000, 0.0000, 0.0000, 0.0156, 0.0000, 0.0000, 0.4609, 0.0000, 0.0078
Min-max-scaled model scores: 0.0195, 0.0078, 0.3398, 0.0000, 0.3398, 0.1914, 0.0781, 0.0000, 0.0000, 0.0000, 0.0000, 0.0195
Meta-classifier scores: 0.0039, 0.0312, 0.3164, 0.0000, 0.0156, 0.0039, 0.3555, 0.0000, 0.0000, 0.0430, 0.2227, 0.0078
-----
Final prediction: Frontal elevation of arms
Class index: 6
Confidence score: 0.3555
```

The output is acceptable because frontal elevation of arms in the data must have had the same board orientation (and no acceleration/movement) as the board being flat on the table. As shown in the serial monitor output, the detection is very stable, and there are not really any visible issues with it besides maybe the confidence score being a bit on the low side. Because of this, the change that would most likely result in higher deployment reliability is simply changing the models to be better.

Question 4: Deployment Behavior on the Arduino

After deploying the ensemble model on the Arduino Nano 33 BLE Sense / Sense Rev2, did you observe any unexpected prediction behavior? For example, did the model repeatedly predict the same class even when the board was moved differently?

Briefly explain why this may happen. In your answer, discuss possible causes such as differences between the original training dataset and the live Arduino sensor data, sensor placement, axis orientation, unit conversion, feature scaling, or mismatch between the motion patterns used during training and testing.

One class that kept showing up was running. No matter which way I oriented the board, any sort of back and forth motion or repeated circular motion resulted in the model predicting "running" with a relatively high accuracy. This was pretty unexpected, as there are other classes like walking, cycling, and jogging that also involve repeated motions. Something that might have caused this is that these classes may be very difficult to distinguish from each other, especially jogging and running. In addition, it is possible that the model is doing a lot of guessing when the board is oriented in a different direction than it is meant to be. Also, when I am testing this, there is a motion mismatch between my own created data for testing and the data the model was trained on. This is likely the biggest factor in why the model can be inconsistent or inaccurate at times.